

UR5e Control Analysis

Part 2 — Independent Simulation, ROS2 Integration, and Runtime Validation

Direct-Effort Cross-Validation, ROS2/C++ Deployment, MPC Runtime Sensitivity, and Practical Execution Assessment

Sooyong Kim

Robotics Engineering Portfolio Project

October 2026

Scope. Part 2 extends the analytical controller study established in Part 1B through two increasingly independent implementation layers. First, PID, computed torque, discrete LQR, and constrained MPC are transferred to an Isaac Sim UR5e through a direct effort interface to test whether the analytical closed-loop regimes survive replacement of the custom plant. Second, PID, LQR, and MPC are deployed as a ROS2/C++ validation package with explicit experiment management, logging, and simulation-time indexing. The ROS2 stage is not a global controller re-optimization exercise. Its purpose is to expose implementation effects that are invisible in the analytical model, quantify runtime behavior, and identify a practical MPC configuration that remains executable in the tested ROS2/Isaac environment.

Contents

Executive Summary	3
1 Relationship to Part 1B and Validation Philosophy	4
1.1 Part 1B controller baselines	4
1.2 Why Part 2 is not another tuning study	4
2 Common Benchmark and Experimental Boundaries	5
3 Part 2A — Independent Direct-Isaac Validation	5
3.1 Purpose	5
3.2 Isaac Sim environment and effort interface	5
3.3 Physics-period mismatch	6
3.4 Pre-physics versus post-physics actuation	6
3.5 Direct-Isaac cross-controller baseline	7
4 Transition to ROS2 Runtime Validation	7
4.1 ROS2 experiment scope	7
4.2 System architecture	8
4.3 Simulation-time MPC indexing	8
5 ROS2 PID Validation	8
6 ROS2 LQR Validation	9
7 MPC Baseline from Part 1B	11
8 Runtime Failure Mechanism of the Model-Selected MPC	11
8.1 Observed sequence	12
8.2 Aggressive effort as the practical trigger	14
9 MPC Runtime Sensitivity Study	14
9.1 Tested configurations	14
9.2 Interpretation of ρ_u	16
9.3 Interpretation of N_c	16
9.4 Practical runtime configuration	16
10 Why the Runtime-Adjusted MPC Was Not Re-Cross-Validated as a New Part 1B Optimum	18

11 ROS2 Runtime Outcome Summary	19
12 Repeatability, Deadline Metrics, and Evidence Quality	20
13 Engineering Findings	20
14 Limitations and Non-Claims	21
15 Reproducibility Map	22
16 Next Stage — Task-Oriented Optimization	22
17 Conclusion	23
A Recorded ROS2 Runtime Summary	23
B Finalization Checklist	24

Executive Summary

Part 1B treated controller comparison as an engineering diagnosis problem rather than a ranking exercise. A common 4 s quintic trajectory, 2 ms controller period, validated nonlinear UR5e plant, and adopted simulation torque constraints were used to compare feedback-only PID, computed torque, discrete LQR, and constrained linear MPC. Part 1B established both controller behavior and implementation baselines: PID exposed gravity-driven tracking error, computed torque demonstrated matched-model cancellation and its sensitivity to model mismatch, LQR required reference-dependent gravity feedforward, and MPC required explicit offline/online algebraic separation to reach a C++/OSQP mean runtime of approximately 0.9605 ms. The final Part 1B MPC used $N_p = 160$, $N_c = 40$, and $\rho_u = 10$.

Part 2 asks a different question: *what survives when the controller is moved outside the model used to design it?* The study therefore uses two validation layers. The first is a direct Isaac Sim validation baseline. The analytical plant is removed from the state-propagation loop, while the same controller definitions drive an independently implemented Isaac articulation through direct joint efforts. This stage exposed two important integration effects before ROS2 was introduced. A 60 Hz/500 Hz physics/controller mismatch invalidated the benchmark, and applying computed-torque commands after the physics step produced a large apparent model discrepancy. Correcting the physics timestep and moving actuation to the pre-physics phase restored close agreement with the Part 1B regime.

The second layer introduces ROS2 Humble and a C++ runtime package. The package separates controller execution, experiment-state management, logging, and Isaac initialization. PID, LQR, and MPC were tested in this ROS2 stage. Computed torque was **not** included in the ROS2 experiment set and no ROS2 computed-torque result is claimed in this report.

The most important new finding concerns MPC. The model-selected $N_p = 160$, $N_c = 40$, $\rho_u = 10$ configuration had already shown good analytical and direct-Isaac behavior. In ROS2, however, the initial low-millisecond solve regime did not persist. The controller issued aggressive corrective effort, the simulated plant accumulated position/velocity error, torque constraints became active, solver cost grew sharply, controller updates were missed, and the closed loop entered a self-reinforcing degradation regime. Only 159 of the nominal 2000 logged runtime updates were retained in the $\rho_u = 10$, $N_c = 40$ run.

A limited runtime sensitivity study therefore varied ρ_u and N_c while holding the prediction horizon at $N_p = 160$ (320 ms). Increasing ρ_u from 10 to 120 penalized corrective input more strongly and reduced aggressive torque action. Reducing N_c from 40 to 20 shortened the independently optimized control window from 80 ms to 40 ms while preserving the same 320 ms prediction window. The tested $\rho_u = 120$, $N_c = 20$ configuration completed all 2000 logged updates, produced approximately 2.49° overall joint RMSE, and retained a mean controller computation time of approximately 0.751 ms with a 99th percentile of approximately 1.64 ms.

This is deliberately reported as a **practical runtime configuration**, not as a new optimum. No exhaustive tuning or global optimization of ρ_u and N_c was performed, and the complete Part 1B modeling/Python-C++ cross-validation sequence was not repeated for every runtime-adjusted configuration. That would answer a different research question. The purpose of Part 2 is to identify differences between analytical, independent simulation, and middleware/runtime execution.

The failed and partially completed MPC runs are therefore used as **runtime-failure evidence**, not as controller-performance benchmarks. Their RMSE values describe the trajectories that occurred while the intended 500 Hz closed loop was degrading; they do not represent a valid optimized-MPC performance level and are not used to rank MPC against PID or LQR. Even the completed $\rho_u = 120$, $N_c = 20$ case is treated only as a practical executable setting identified within the tested cases. Task-oriented controller optimization is reserved for the next stage, where a manipulation objective such as pick-and-place can define meaningful trade-offs among tracking

error, execution time, constraint activity, robustness, path quality, and task success.

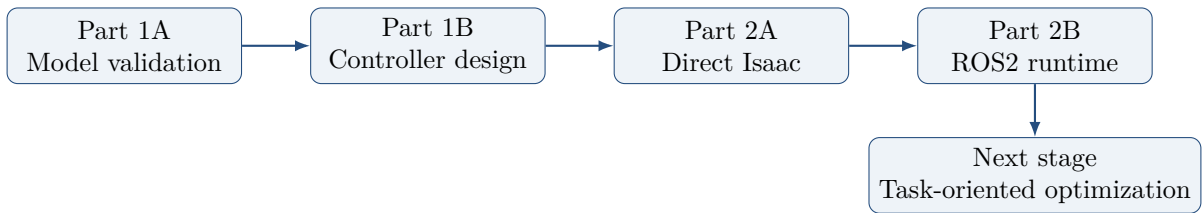
1. Relationship to Part 1B and Validation Philosophy

Part 1B begins from the validated UR5e rigid-body plant

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}) + \mathbf{g}(\mathbf{q}) = \boldsymbol{\tau}, \quad (1)$$

and deliberately holds the benchmark fixed while controller structure changes. The Part 1B objective is explicitly diagnostic: identify what each controller gains, what assumptions it depends on, and what fails first when those assumptions are violated. This philosophy carries directly into Part 2.

The complete project sequence is therefore



The progression matters because each layer removes a different source of ambiguity. Part 1B asks whether the controller behaves as expected on the validated analytical plant. Direct Isaac asks whether those conclusions survive an independently implemented rigid-body simulator. ROS2 then asks whether the same controller cores remain well behaved after middleware, callback timing, message transport, logging, discrete-index management, and C++ execution are introduced.

1.1 Part 1B controller baselines

The common Part 1B benchmark uses

$$\mathbf{q}_0 = [0, -90, 90, -90, -90, 0]^\circ, \quad \mathbf{q}_f = [20, -60, 60, -70, -70, 20]^\circ,$$

a 4 s quintic joint-space trajectory, $T_s = 0.002$ s (500 Hz), and adopted simulation torque constraints

$$\boldsymbol{\tau}_{\max} = [150, 150, 150, 28, 28, 28]^T \text{ N m}.$$

Part 1B intentionally uses a moderate trajectory. Part 1A inverse dynamics showed a nominal J2 peak of approximately 40.79 N m, about 27.2% of the adopted J2 limit, so the nominal controller comparison was not intended to collapse into a saturation experiment.

The Part 1B nominal overall joint RMSE values were 0.469844° for PID, 0.001244° for computed torque, 0.015502° for LQR, and 0.048474° for MPC. These values are not treated as a universal ranking: the four controllers exploit different assumptions.

1.2 Why Part 2 is not another tuning study

Part 1B already contains substantial tuning, runtime profiling, and robustness testing. In particular, the final MPC was selected after prediction-horizon and control-horizon studies, algebraic offline/online decomposition, and Python-C++ torque cross-validation. Repeating full controller design every time an implementation effect appeared in Part 2 would blur the distinction between controller quality and runtime architecture.

Part 2 therefore follows a stricter rule: preserve the controller definition when testing a new implementation layer, diagnose discrepancies first, and modify only what is necessary to study the newly exposed mechanism. The later MPC parameter variations are explicitly a limited runtime sensitivity study, not a replacement for the Part 1B design study.

2. Common Benchmark and Experimental Boundaries

Table 1: Common benchmark retained across the analytical, direct-Isaac, and ROS2 stages.

Item	Definition
Reference	4 s quintic joint-space trajectory
Start pose	$[0, -90, 90, -90, -90, 0]^\circ$
Goal pose	$[20, -60, 60, -70, -70, 20]^\circ$
Nominal controller period	2 ms (500 Hz)
Nominal logged updates	2000 from approximately $t = 0$ through $t = 3.998$ s
Torque limits	$[150, 150, 150, 28, 28, 28]$ N m
Primary state feedback	Joint position and velocity
Primary output	Joint effort command

Important boundary. The adopted torque limits are simulation/imported-model constraints rather than independently verified physical UR5e motor ratings. The project also does not claim hard real-time operation on a real-time operating system or hardware robot.

3. Part 2A — Independent Direct-Isaac Validation

3.1 Purpose

The first Part 2 layer removes the analytical plant from state propagation while avoiding ROS2. The controller still uses the model quantities required by its mathematical definition, but the actual state evolves under Isaac Sim rigid-body physics. Conceptually,

$$(q_d, \dot{q}_d, \ddot{q}_d) \rightarrow \text{controller} \rightarrow \tau \rightarrow \text{Isaac UR5e} \rightarrow (q, \dot{q}).$$

This direct interface provides a baseline that later helps distinguish controller/physics discrepancies from ROS2 integration effects.

3.2 Isaac Sim environment and effort interface

The tests use NVIDIA Isaac Sim 6.1. The articulation is rooted at `/ur5e`; the controlled joints are shoulder pan, shoulder lift, elbow, wrist 1, wrist 2, and wrist 3 in the same order used by the analytical model. Position/velocity drive action was removed from the active control path so that a hidden simulator servo would not contribute to the measured response.

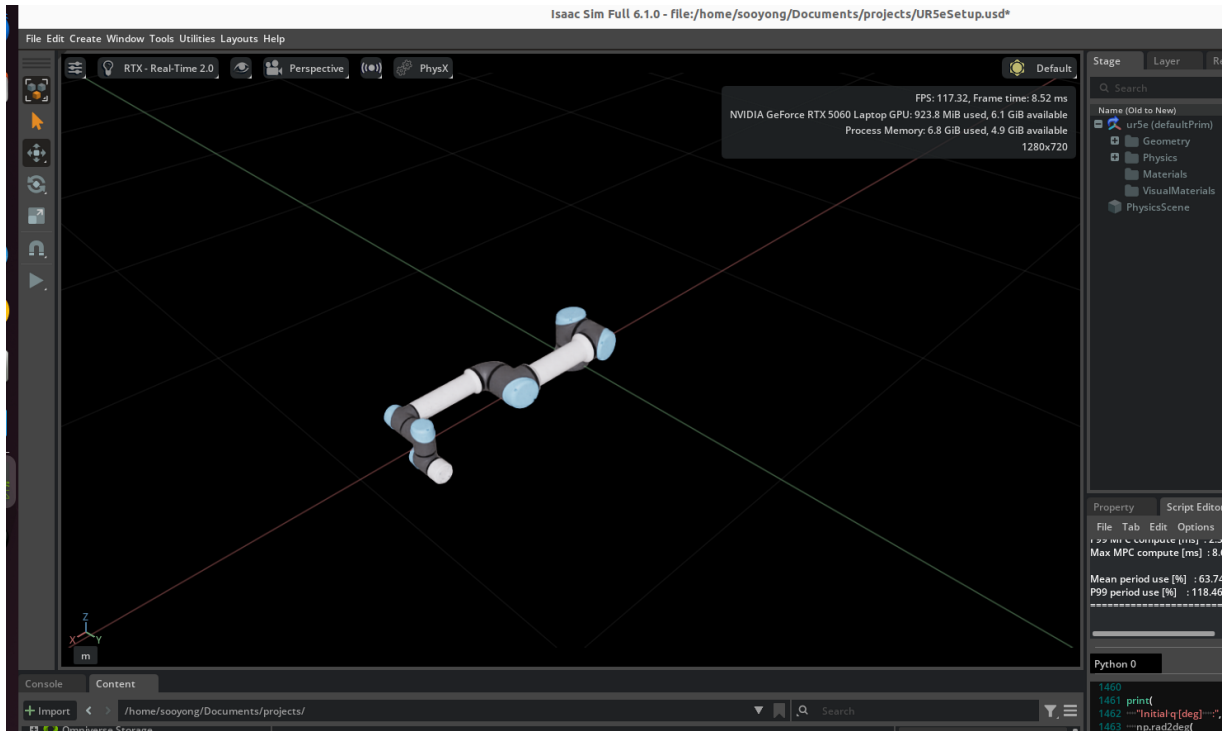


Figure 1: Isaac Sim 6.1 direct-effort validation environment used to establish an independent controller/physics baseline before ROS2 integration.

3.3 Physics-period mismatch

An early PID transfer advanced the controller reference at 2 ms while the physics scene advanced at approximately 16.67 ms (60 Hz). Overall joint RMSE rose to approximately 3.75° , and the first three adopted torque limits were reached. This run was treated as an invalid reproduction of the Part 1B benchmark rather than evidence that the PID controller itself had failed. After the physics timestep was explicitly corrected to 2 ms, the expected Part 1B behavior returned.

The engineering lesson is simple but important: preserving gains without preserving the temporal definition of the plant does not preserve the closed-loop system.

3.4 Pre-physics versus post-physics actuation

Computed torque exposed a second synchronization problem. Applying the unchanged controller after the physics step produced approximately 0.700694° overall RMSE. Moving the same effort application to `PRE_PHYSICS_STEP` reduced overall RMSE to approximately 0.00082685° , an improvement of roughly $847\times$. No controller equations, gains, trajectory definitions, or torque limits were changed.

This result is retained because it demonstrates a recurring Part 2 theme: an implementation-phase error can masquerade as a dynamics-model error.

3.5 Direct-Isaac cross-controller baseline

Table 2: Part 1B analytical benchmark versus direct Isaac validation.

Controller	Part 1B RMSE [deg]	Direct Isaac RMSE [deg]	Difference [deg]
PID	0.469844	0.464275	-0.005569
Computed torque	0.001244	0.000827	-0.000417
LQR	0.015502	0.014925	-0.000577
MPC	0.048474	0.048790	+0.000316

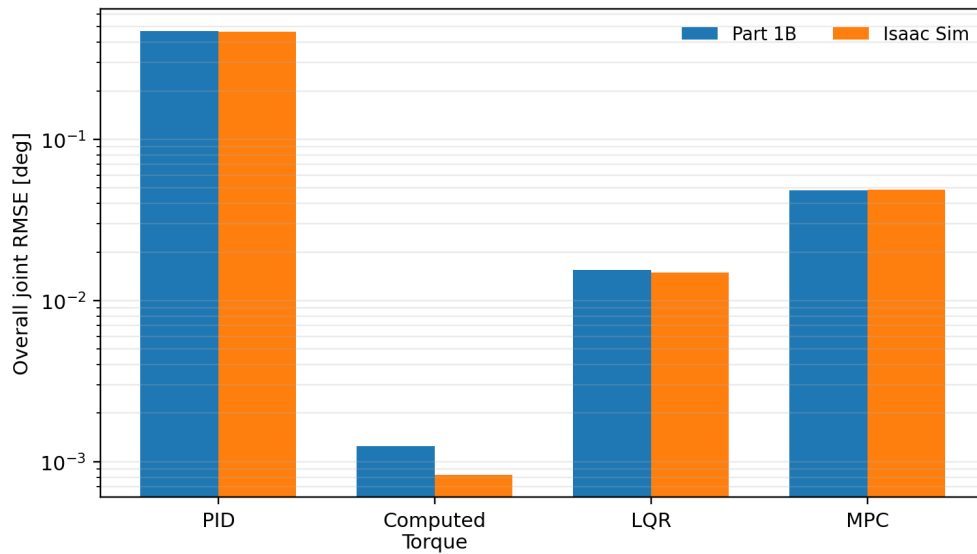


Figure 2: Overall joint RMSE from the Part 1B analytical benchmark and direct Isaac validation. This direct baseline establishes that the controller regimes survive replacement of the custom analytical plant when timing and actuation phase are configured correctly.

The direct validation includes all four Part 1B controllers. Computed torque is intentionally retained in this section because it was actually tested at the direct-Isaac layer and provides a strong matched-model consistency check. It is not carried into the later ROS2 experiment set.

4. Transition to ROS2 Runtime Validation

The direct-Isaac layer intentionally avoids middleware. That isolation is useful, but it also removes the communication and scheduling mechanisms that a deployable ROS2 architecture must handle. The next validation layer therefore preserves the tested controller cores and changes the interface architecture.

4.1 ROS2 experiment scope

The ROS2 runtime experiment evaluates three controllers:

- PID,
- discrete LQR,

- constrained linear MPC.

Computed torque was **not tested in the ROS2 runtime experiment**. Its absence from the ROS2 comparison tables is therefore intentional and should not be interpreted as a negative result.

4.2 System architecture

The final package uses ROS2 Humble and Isaac Sim 6.1. Controller execution, initialization, lifecycle management, and logging are separated so that each failure mode can be inspected independently.



Figure 3: ROS2/Isaac validation architecture. The controller core is separated from experiment sequencing, initialization, and logging.

The Isaac initializer forces the benchmark start pose q_0 and zero joint velocity while HOLD is active. The validation manager verifies the initial joint pose over consecutive samples, releases HOLD, starts the 4 s RUNNING phase, commands zero effort after completion, allows a short free-fall phase, and terminates the launch. The logger records measured q , measured $\dot{q}$, reconstructed q_{ref} and $\dot{q}_{ref}$, commanded joint torque, simulation time, and controller computation time.

4.3 Simulation-time MPC indexing

MPC is indexed using Isaac simulation time rather than assuming that every wall-clock callback arrives exactly every 2 ms. Duplicate or older discrete indices are ignored. If computation is late and one or more discrete indices are missed, the controller jumps to the current simulation-time index rather than replaying obsolete MPC steps.

This policy prevents backlog replay from making the reference trajectory itself lag behind the simulated plant. It also creates a useful runtime diagnostic: a sparse logged sequence is evidence that the runtime could not service every nominal 2 ms control opportunity.

5. ROS2 PID Validation

PID provides the simplest runtime baseline because it does not solve an optimization problem and does not explicitly use the rigid-body model online. The ROS2 run completed all 2000 logged updates over approximately 3.998 s.

The measured overall joint RMSE was approximately

$$1.006^\circ,$$

with a maximum absolute joint error of approximately 5.16° . The controller computation time was

negligible relative to the 2 ms period: mean approximately 0.00164 ms, 99th percentile approximately 0.00416 ms, and maximum approximately 0.0362 ms.

Unlike the direct-Isaac baseline, the ROS2 PID run reached the adopted J2/J3 torque limits. This is a meaningful implementation-layer difference and is not hidden by reporting only RMSE.

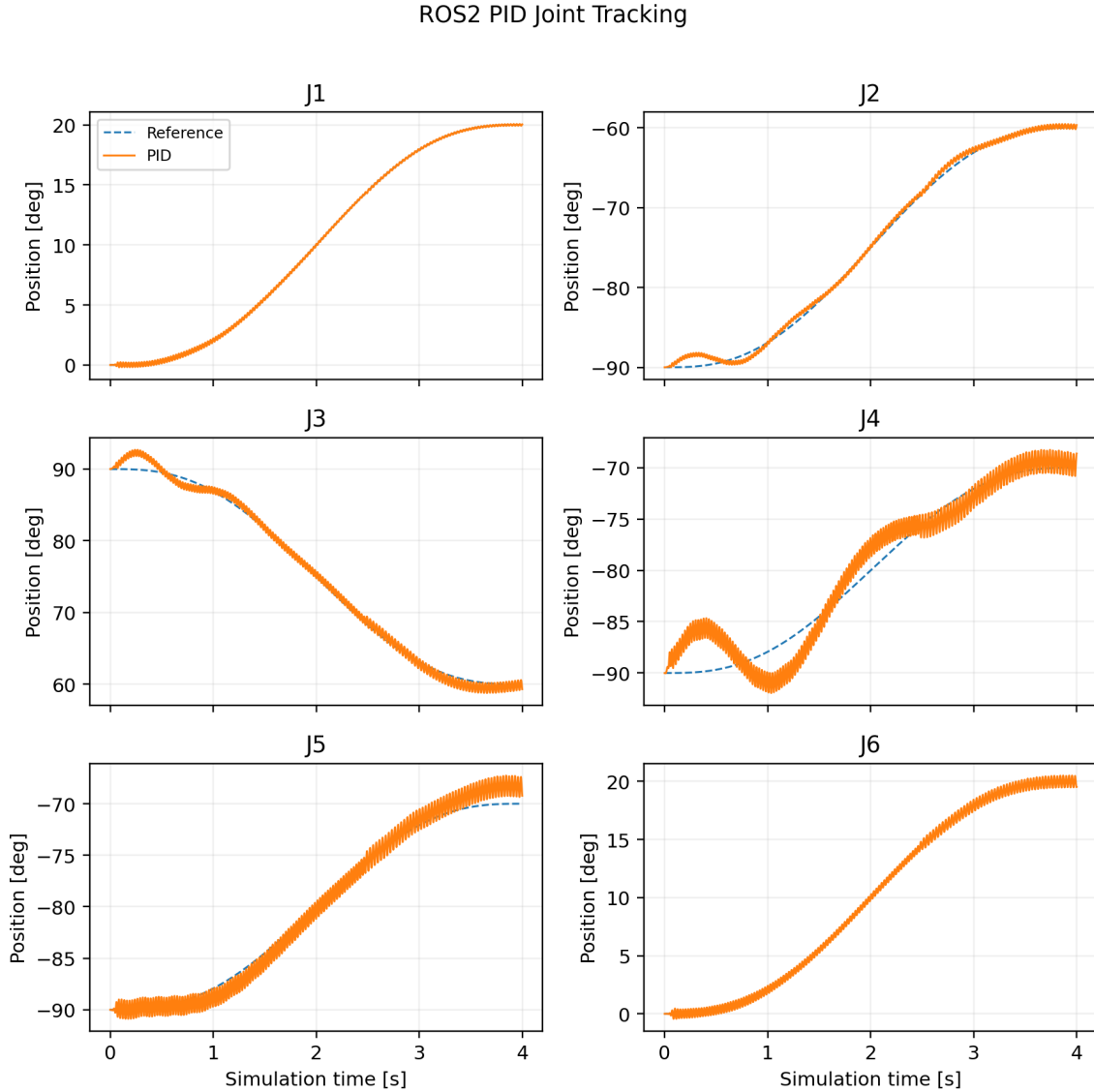


Figure 4: ROS2 PID joint tracking over the 4 s benchmark.

6. ROS2 LQR Validation

The ROS2 LQR implementation uses the Part 1B fixed start-equilibrium linearization and reference-dependent gravity feedforward. The run completed all 2000 logged updates.

The measured overall joint RMSE was approximately

$$2.344^{\circ},$$

with a maximum absolute joint error of approximately 6.22° . Mean computation time was approximately 0.00728 ms, the 99th percentile was approximately 0.0151 ms, and the maximum

was approximately 0.0386 ms. No controller-computation deadline miss occurred under the 2 ms threshold.

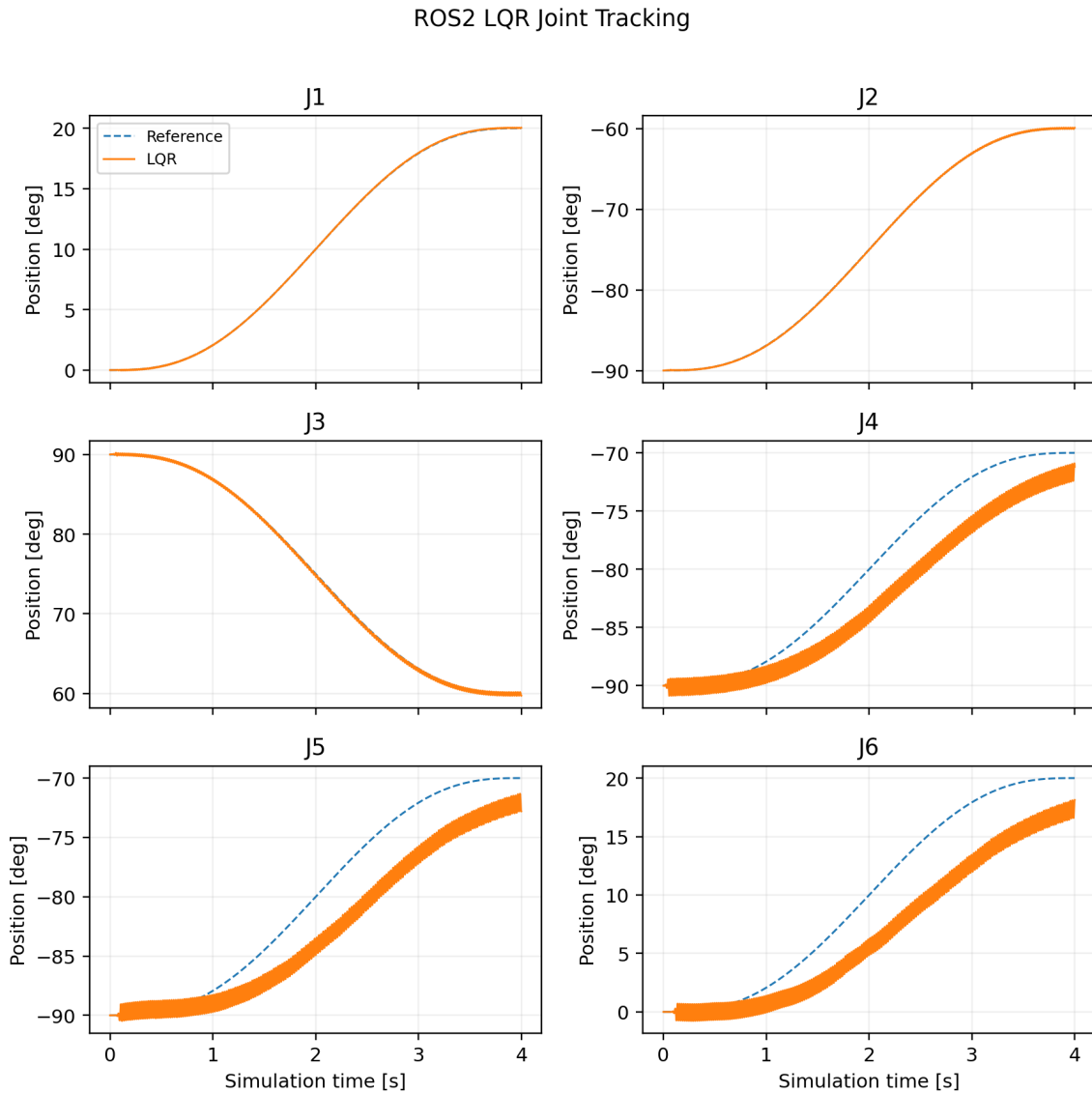


Figure 5: ROS2 LQR joint tracking over the 4 s benchmark. The runtime computation is extremely light; the larger error relative to the direct-Isaac baseline therefore cannot be attributed to LQR arithmetic cost alone.

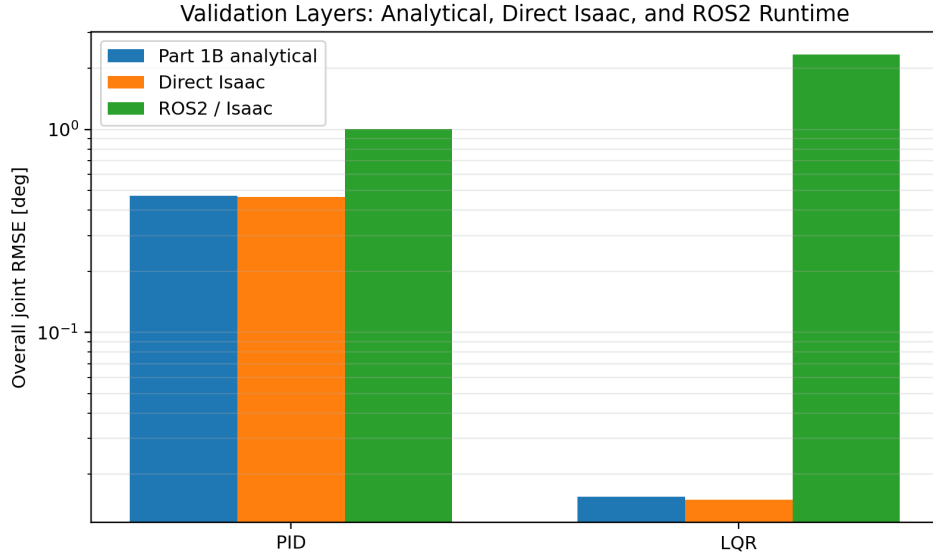


Figure 6: PID and LQR overall joint RMSE across the Part 1B analytical, direct-Isaac, and ROS2/Isaac layers. Only controller definitions that are directly comparable across all three layers are shown. The plot is diagnostic rather than a controller ranking.

7. MPC Baseline from Part 1B

The Part 1B constrained linear MPC uses the same discrete local model as LQR but optimizes a finite sequence of torque corrections. In equilibrium coordinates,

$$z_{k+1} = A_d z_k + B_d v_k. \quad (2)$$

The trajectory-dependent gravity term is included as a known affine feedforward shift. The Part 1B final design uses

$$N_p = 160, \quad N_c = 40, \quad \rho_u = 10,$$

which corresponds to a 320 ms prediction window and an 80 ms control window at a 2 ms sample period.

Part 1B had already treated implementation as part of MPC design. The first dense Python benchmark required approximately 17.97 ms per update. Profiling showed that fixed and reference-known algebra was being recomputed online. Rewriting the QP gradient into a state-dependent term plus a precomputed reference term reduced the optimized Python update to approximately 1.03–1.08 ms. C++/Eigen/OSQP then reproduced the Python torque sequence with a maximum absolute difference of approximately 2.03×10^{-10} Nm and a mean runtime of approximately 0.9605 ms. This established that the C++ solver implementation matched the Part 1B numerical formulation under the same supplied data.

Direct Isaac subsequently showed that this $\rho_u = 10$, $N_c = 40$ controller could reproduce the Part 1B nominal regime when driven through the direct simulator interface. The ROS2 failure described next therefore cannot be reduced to “the MPC equations were wrong” or “the C++ port did not match Python.”

8. Runtime Failure Mechanism of the Model-Selected MPC

8.1 Observed sequence

The ROS2 $\rho_u = 10$, $N_c = 40$ run did not simply begin as a slow optimization. After startup overhead, early updates were still in the low-millisecond regime, broadly consistent with the Part 1B C++ implementation order of magnitude. The more important failure was closed-loop interaction.

The recorded sequence is interpreted as follows:

1. The low input penalty allows relatively aggressive corrective torque.
2. The Isaac/ROS2 closed loop does not reproduce the idealized response closely enough to keep the state near the nominal predicted trajectory.
3. Position and especially velocity error accumulate.
4. Torque constraints become active and the QP moves into a more difficult operating region.
5. Computation time grows sharply; missed 2 ms opportunities become frequent.
6. Because the MPC is indexed by simulation time, missed control opportunities are not replayed. The state therefore evolves farther from the nominal prediction between successful solves.
7. Larger state error demands still stronger correction, reinforcing the degradation.

The CSV does not retain OSQP iteration count, so this report does not present an iteration-count time history as a quantitative result. Runtime debugging showed solver effort increasing during the failing regime, but the preserved quantitative evidence in this draft is computation time, state error, torque utilization, and missing update density.

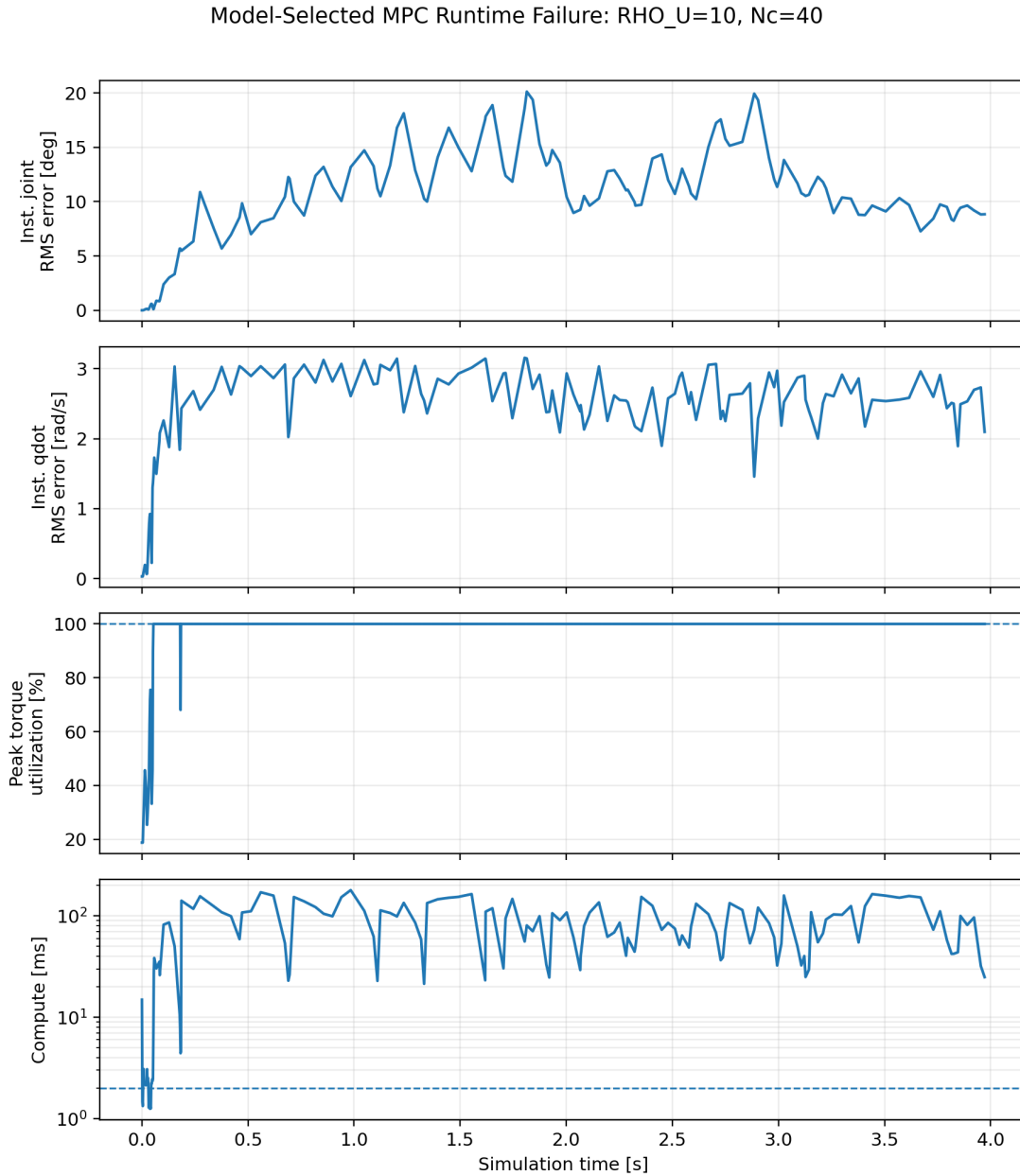


Figure 7: Time evolution of the model-selected MPC ($\rho_u = 10$, $N_c = 40$) in ROS2. Tracking/velocity error grows, adopted torque limits become active, and computation time rises by orders of magnitude. The sparse simulation-time record later in the run reflects missed nominal 2 ms control opportunities.

The run retained only 159 logged controller updates, or 7.95% of the nominal 2000 updates. Overall joint RMSE over the retained samples was approximately 10.69° , and peak torque utilization reached the adopted limit. Mean recorded computation time was approximately 69.8 ms, with a 99th percentile of approximately 166.8 ms and a maximum of approximately 179.3 ms.

These large aggregate timing numbers describe the failed regime; they should not be read as the intrinsic cost of the MPC formulation. Part 1B already demonstrated sub-millisecond mean C++ execution under the model-side benchmark. The Part 2 result instead shows that solver cost can become state- and constraint-regime dependent once the controller, plant response, and runtime timing interact.

8.2 Aggressive effort as the practical trigger

The practical issue is not merely torque magnitude but the aggressiveness of corrective action. The current MPC formulation constrains torque amplitude but does not impose an explicit torque slew-rate constraint. Increasing ρ_u therefore became a direct way to penalize corrective effort more strongly without redesigning the full MPC formulation.

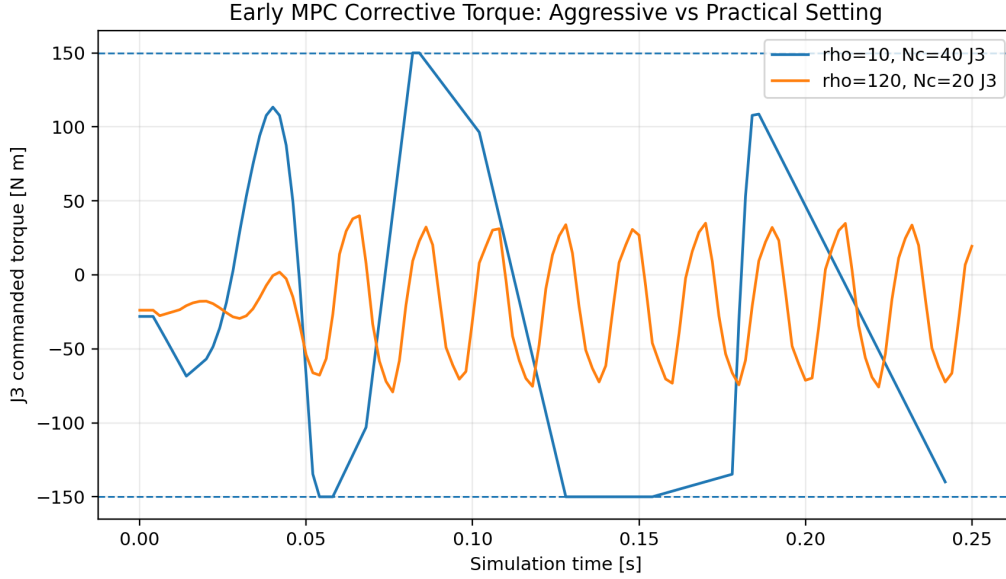


Figure 8: Early J3 torque command comparison. Increasing the input penalty reduces the aggressive correction relative to the model-selected $\rho_u = 10$ setting. This is a runtime-oriented adjustment, not a proof of optimal tuning.

An explicit Δu penalty or torque-rate/slew-rate constraint is a natural future extension. It was not added here because the objective of Part 2 was to diagnose implementation transfer and identify a practical executable region, not to launch a new full MPC formulation study.

9. MPC Runtime Sensitivity Study

9.1 Tested configurations

The prediction horizon was held fixed at $N_p = 160$, preserving the 320 ms future state window. Two parameters were varied:

- ρ_u : input-weight scaling in the MPC Hessian. Increasing ρ_u makes corrective input more expensive and reduces aggressiveness.
- N_c : number of independently optimized control moves. At 2 ms sampling, $N_c = 40$ corresponds to an 80 ms control window and $N_c = 20$ to a 40 ms control window.

The four tested ROS2 configurations were $\rho_u/N_c = 10/40$, $10/20$, $120/40$, and $120/20$.

Table 3: ROS2 MPC runtime sensitivity results from the recorded CSV data. RMSE values from incomplete runs characterize the observed failure trajectories; they are not treated as valid controller-performance benchmarks.

ρ_u / N_c	Samples	Complete [%]	RMSE [deg]	Mean [ms]	P99 [ms]	Max [ms]	> 2 ms [%]
10 / 40	159	7.95	10.694	69.755	166.800	179.316	94.97
10 / 20	186	9.30	14.181	62.155	160.036	163.850	95.16
120 / 40	1730	86.50	3.389	3.206	36.941	70.423	46.99
120 / 20	2000	100.00	2.486	0.751	1.636	2.901	0.25

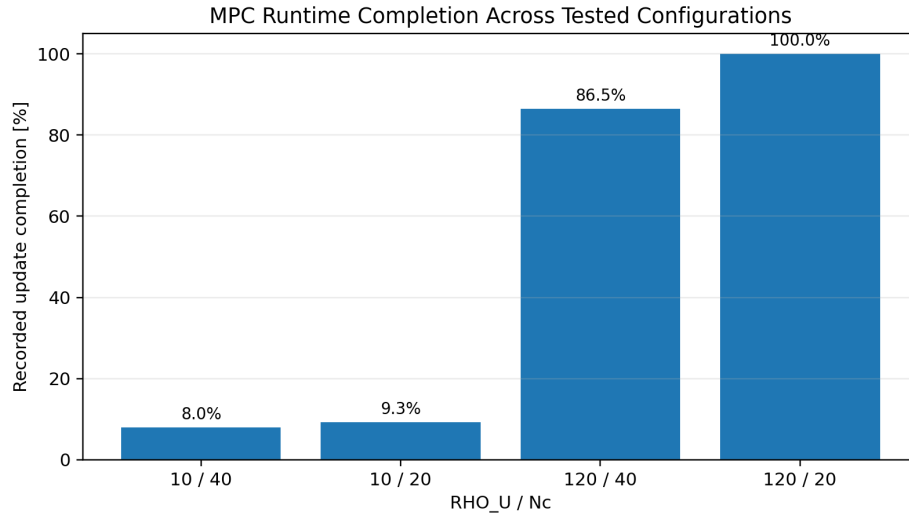


Figure 9: Logged update completion ratio for the four tested MPC configurations. The strongest change occurs after increasing the input penalty; reducing the control horizon to 40 ms then yields a fully completed run in the tested data.

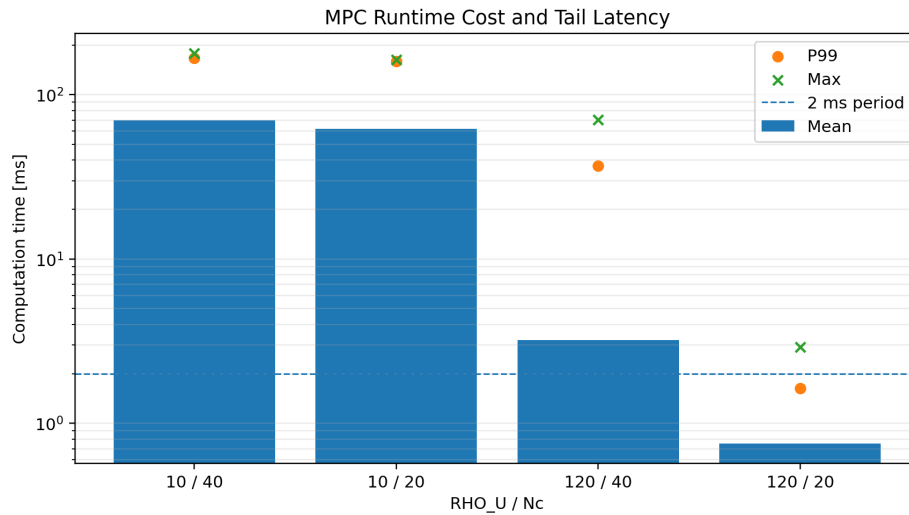


Figure 10: MPC mean, 99th-percentile, and maximum computation time. The logarithmic scale emphasizes tail latency and the large difference between failing and practical execution regimes.

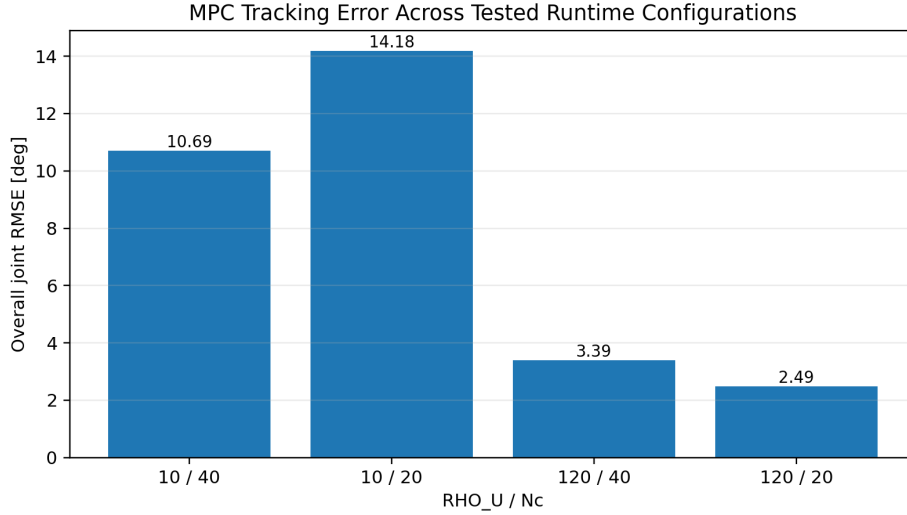


Figure 11: Overall joint RMSE across the tested MPC runtime configurations. For incomplete configurations, RMSE describes the retained failure trajectory rather than sustained MPC tracking capability. The values are not an optimization surface or a controller ranking.

9.2 Interpretation of ρ_u

The largest qualitative change is associated with increasing ρ_u from 10 to 120. This does more than reduce an abstract cost-function term: it changes the physical control behavior by making corrective torque more expensive. In the tested runtime, that reduced aggressive input action, helped keep the plant closer to the predicted regime, reduced sustained constraint activity, and prevented the solver from spending most of the run in the pathological high-cost state seen with $\rho_u = 10$.

This causal interpretation is intentionally stated as a closed-loop engineering interpretation rather than as a formal proof. The experiment changes both controller behavior and the subsequent state trajectory, so solver workload is an outcome of the entire controller–plant–runtime interaction.

9.3 Interpretation of N_c

Reducing N_c from 40 to 20 does *not* shorten the prediction horizon. The controller continues to predict 320 ms ahead through $N_p = 160$. What changes is the independently parameterized control window: 80 ms for $N_c = 40$ versus 40 ms for $N_c = 20$.

In the recorded $\rho_u = 120$ cases, $N_c = 20$ completed all updates, whereas $N_c = 40$ retained 1730/2000 updates and eventually reached the adopted torque limits. The $N_c = 20$ case also had a much smaller tail-latency distribution. This is meaningful evidence for the tested runtime, but it is not sufficient to claim that $N_c = 20$ is universally superior to $N_c = 40$.

9.4 Practical runtime configuration

The configuration

$$\boxed{N_p = 160, \quad N_c = 20, \quad \rho_u = 120}$$

is therefore reported as the practical runtime setting identified in this study.

Only this completed $\rho_u = 120$, $N_c = 20$ run is treated as a valid full-trajectory MPC runtime case in the final ROS2 outcome summary. The other three MPC data sets are retained because they

reveal how the runtime deteriorates as the controller leaves the executable region; they are not interpreted as alternative full-run MPC performance results.

Practical MPC Runtime: $\text{RHO}_U=120$, $N_c=20$

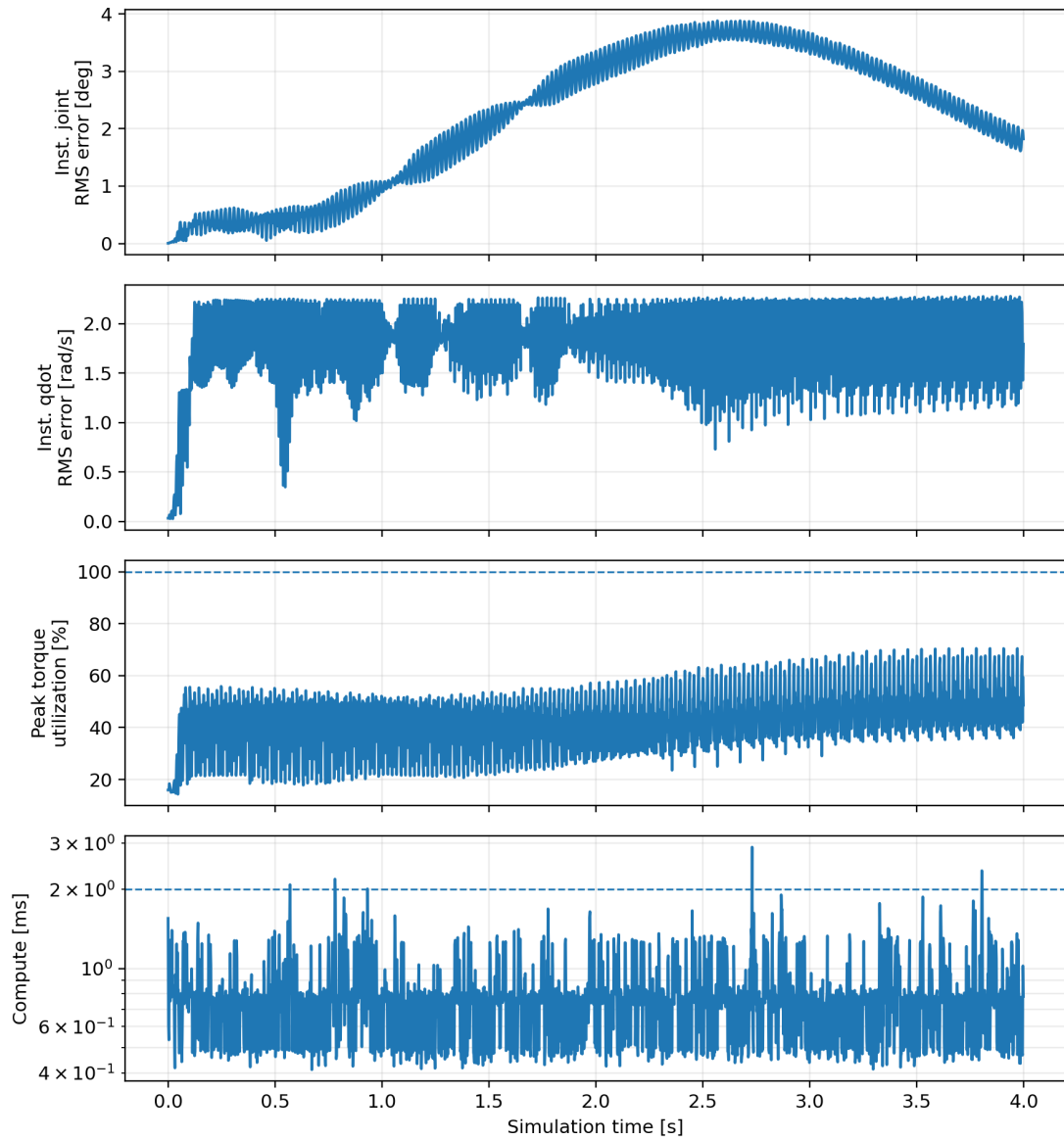


Figure 12: Practical MPC runtime configuration. The full 4 s simulation-time trajectory is retained, peak torque utilization remains below the adopted limits, and controller computation stays predominantly below the nominal 2 ms period.

ROS2 Practical MPC Joint Tracking

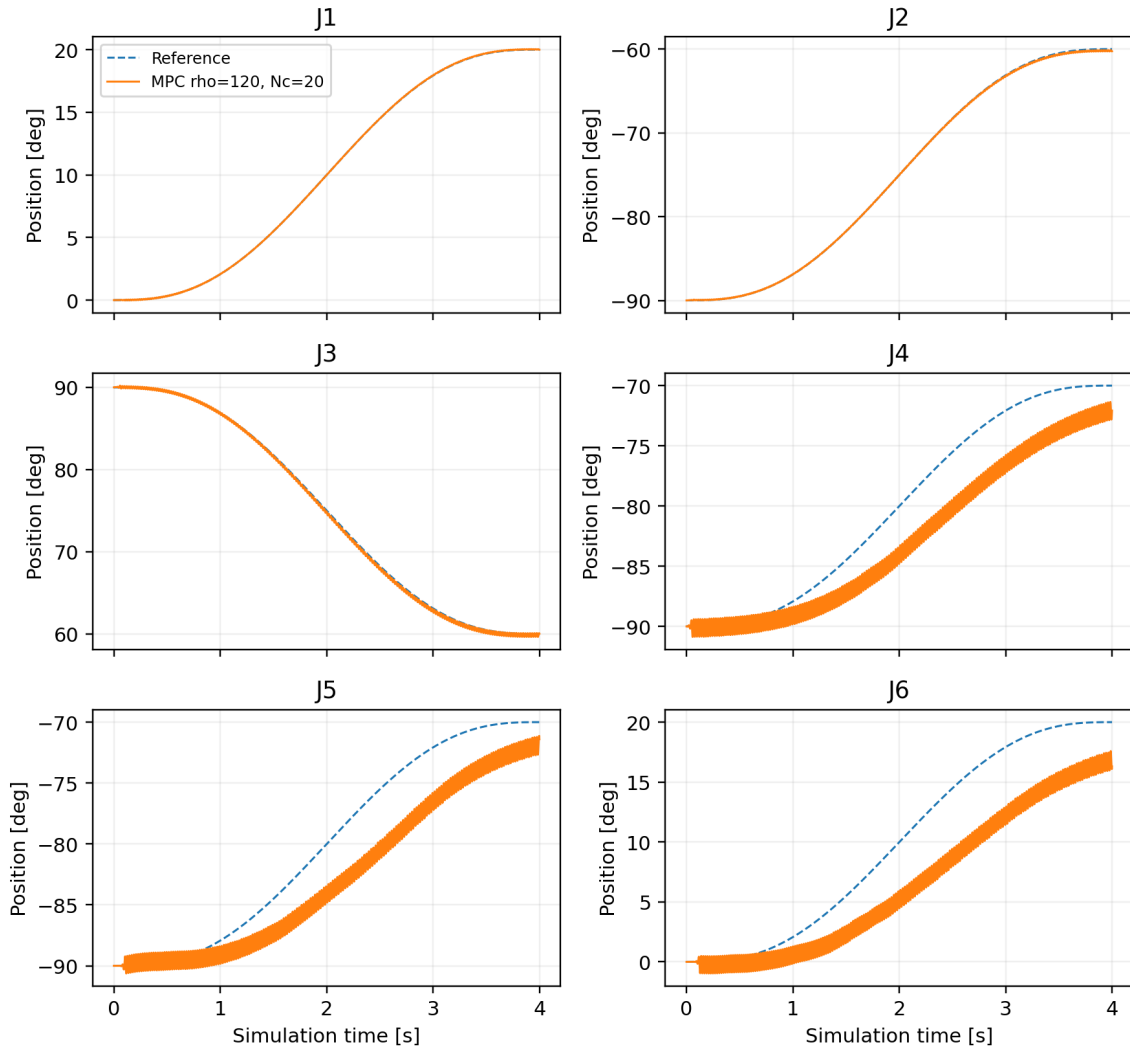


Figure 13: ROS2 joint tracking for the practical MPC configuration $\rho_u = 120$, $N_c = 20$.

10. Why the Runtime-Adjusted MPC Was Not Re-Cross-Validated as a New Part 1B Optimum

The $\rho_u = 120$, $N_c = 20$ setting changes the MPC design parameters relative to the final Part 1B configuration. A complete new controller-design claim would therefore require repeating the relevant analytical sweeps, Python nonlinear benchmarks, numerical cross-validation, and robustness studies. That work was deliberately not performed here.

The reason is scope. Part 2 is designed to compare analytical, independent-simulation, and ROS2 runtime behavior, not to optimize a new controller every time the runtime reveals an implementation sensitivity. The limited ρ_u/N_c sweep answers a narrower question: *is there a nearby practical region in which the same MPC architecture remains executable under the tested runtime?*

Accordingly:

- $\rho_u = 120$, $N_c = 20$ is **not** called globally optimal.
- The four runtime cases are **not** presented as an exhaustive tuning surface.

- No new robustness claim is made for the runtime-adjusted setting.
- No claim is made that the setting would remain best for a different trajectory or task.

This boundary is important because it preserves the logic of the project. Part 1B is where controller design and robustness are studied systematically. Part 2 is where implementation transfer is diagnosed.

11. ROS2 Runtime Outcome Summary

Table 4: Descriptive ROS2 runtime outcomes for the three controllers that completed the final experiment set. MPC uses the practical $\rho_u = 120$, $N_c = 20$ setting. Computed torque was not included in the ROS2 experiment set. The table is not a controller-performance ranking.

Controller	Samples	RMSE [deg]	Max err. [deg]	Mean [ms]	P99 [ms]	Peak util. [%]
PID	2000	1.006	5.164	0.00164	0.00416	100.0
LQR	2000	2.344	6.223	0.00728	0.01510	72.82
MPC practical	2000	2.486	6.685	0.75066	1.63587	70.45

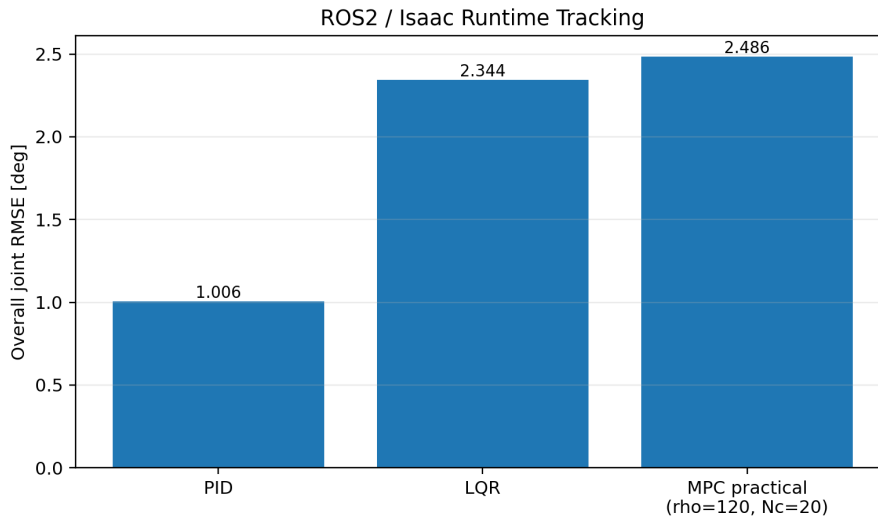


Figure 14: Recorded overall joint RMSE for the three completed ROS2 runtime cases. These values document the specific runs; they are not used to infer a controller ranking or an optimized performance limit.

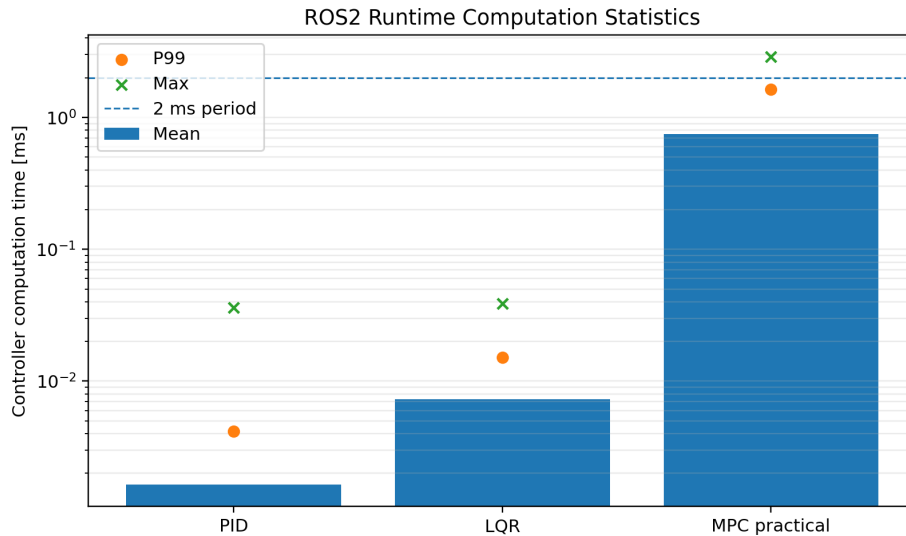


Figure 15: Mean, P99, and maximum controller computation time for the three ROS2 controllers. PID and LQR are computationally negligible relative to the 2 ms period; practical MPC is substantially heavier but retains a sub-2 ms P99 in the recorded run.

The final ROS2 table is intentionally descriptive. It should not be used to infer that PID outperformed LQR or that LQR outperformed MPC as a controller design. Part 2 did not retune each controller to a common ROS2 optimum, and the practical MPC case was selected for executable runtime behavior rather than for minimum tracking error. The failed MPC configurations are excluded from this summary because they did not sustain the intended full-rate closed loop.

Accordingly, cross-controller performance optimization is deferred to the next task-oriented stage. A defined manipulation task will provide a meaningful common objective under which tracking quality, execution time, path quality, constraint activity, robustness, and task success can be assessed together.

12. Repeatability, Deadline Metrics, and Evidence Quality

Runtime control should not be judged only by mean computation time. The preserved CSV data therefore support several complementary metrics: full-trajectory completion, simulation-time gaps, mean/P95/P99/max computation time, deadline miss rate, tracking error, and torque utilization.

The $\rho_u = 120$, $N_c = 40$ case is particularly instructive. A previous development run appeared much closer to the $N_c = 20$ result, but the repeated run preserved here completed only 86.5% of nominal updates and showed substantial timing tails. This run-to-run discrepancy is itself evidence that near-boundary configurations can be sensitive to execution conditions.

For the portfolio version of the project, the retained CSV files are treated as the authoritative final data set. A more formal repeatability study would execute multiple independent runs per configuration and report distributions of completion ratio, RMSE, P99 latency, and saturation incidence. That experiment is not yet claimed as complete in this draft.

13. Engineering Findings

1. **Controller transfer requires timing transfer.** A controller designed for a 2 ms plant/controller definition is not being tested under the same closed-loop system if the simulator physics runs

near 60 Hz.

2. **Actuation phase can masquerade as model error.** Moving computed-torque effort application from post-step to pre-step reduced direct-Isaac RMSE by roughly $847\times$ without changing controller mathematics.
3. **Independent physics validation and runtime validation are different tests.** Direct Isaac established that the controller equations and plant-side behavior could agree closely without ROS2. The ROS2 layer then exposed integration effects that the direct interface deliberately removed.
4. **A model-selected MPC can be runtime-fragile even when its nominal arithmetic is fast.** The $\rho_u = 10$, $N_c = 40$ solver did not simply start “too slow.” Aggressive control action drove the closed loop into a state/constraint regime where solver cost and missed updates increased dramatically.
5. **Input weighting has a physical runtime consequence.** Increasing ρ_u reduced aggressive corrective torque and helped keep the runtime trajectory inside a tractable operating region.
6. **Prediction horizon and control horizon should not be conflated.** The practical configuration retained the 320 ms prediction window while shortening the independently optimized control window from 80 ms to 40 ms.
7. **Tail latency and completion ratio matter more than mean runtime alone.** A controller can have an acceptable initial or average solve time while still failing to service enough 2 ms opportunities to preserve the intended closed loop.
8. **Torque-amplitude constraints do not directly regulate torque slew.** The observed aggressive input variation motivates explicit Δu regularization or torque-rate constraints in a future MPC formulation.
9. **A practical runtime setting is not automatically an optimal controller.** The $\rho_u = 120$, $N_c = 20$ result identifies an executable region within the tested cases; it does not replace the systematic Part 1B design/robustness study.
10. **Failure-run tracking error is not a controller-performance metric.** Once an MPC configuration no longer services the intended 2 ms control sequence, its retained RMSE describes the degraded runtime trajectory rather than the achievable tracking performance of a properly executing MPC. Those runs are used to diagnose failure, not to rank controllers.

14. Limitations and Non-Claims

The following boundaries are intentional:

- The work uses simulation, not physical UR5e hardware.
- No hard-real-time operating system, deterministic ROS2 executor, or hardware deadline guarantee is demonstrated.
- The adopted torque limits are simulation/imported-model constraints rather than independently verified motor ratings.
- Only one common 4 s benchmark trajectory is used for the final runtime comparison.
- Computed torque was validated in the direct-Isaac layer but was not included in the ROS2 runtime experiment.
- The MPC runtime sensitivity study varies only ρ_u and N_c over four selected cases; it is not a global parameter optimization.

- The runtime-adjusted MPC was not re-run through the complete Part 1B analytical robustness and Python–C++ cross-validation workflow.
- RMSE from incomplete MPC runs is reported only as descriptive failure data and is not used for cross-controller performance comparison.
- No conclusion is drawn from the numerical ordering of PID, LQR, and practical-MPC RMSE in the ROS2 runs; controller optimization under a common task objective is deferred to the next stage.
- OSQP iteration count is not retained in the final CSV files, so this report does not present iteration count as a quantitative time-history metric.
- Multiple independent repeats per final configuration were not systematically collected; repeatability is therefore discussed qualitatively rather than claimed statistically.
- No explicit torque-rate constraint is present in the current MPC formulation.

15. Reproducibility Map

Purpose	Artifact
Part 1B analytical controller design	part1_modeling_control_analysis/
Part 1B technical report	reports/UR5e_Part_1B_Controller_Design_and_Robustness_Assessment
ROS2 controller node	part2_ros2_isaac_validation/ur5e_ros2_control/src/controller_r
Validation manager	.../src/validation_manager.cpp
CSV logger	.../src/logger_node.cpp
Isaac initialization/HOLD script	.../scripts/isaac_initializer.py
ROS2 PID result	.../results/pid_validation.csv
ROS2 LQR result	.../results/lqr_validation.csv
MPC runtime sensitivity data	.../results/mpc_rho*_nc*.csv
Figure-generation script in this report bundle	scripts/generate_figures.py
Derived runtime summary	data/ros2_summary.csv

16. Next Stage — Task-Oriented Optimization

The next stage should not simply repeat tuning against the same 4 s benchmark. A robotic task provides a more meaningful objective. A pick-and-place demonstration is a natural continuation because it introduces motion planning, collision constraints, grasp/placement geometry, task sequencing, and execution success on top of the controller/runtime stack already validated here.

Under a task-oriented study, controller and planning choices can be assessed using a broader objective set:

- trajectory tracking error,
- task completion and success rate,
- execution time,
- path smoothness and followability,
- torque magnitude and torque-rate activity,

- constraint activation,
- robustness to payload/object variation,
- runtime deadline behavior,
- recovery from task-level disturbances or replanning events.

This is the appropriate stage for renewed optimization. The question would no longer be “which controller best reproduces the common benchmark?” but “which controller/planner configuration best completes a defined robotic task under explicit performance and constraint criteria?”

17. Conclusion

Part 2 extends the Part 1B analytical controller study through two independent implementation layers. Direct Isaac validation first established that the principal Part 1B controller regimes survive replacement of the custom analytical plant when the physics timestep and actuation phase are configured consistently. That layer also supplied a baseline for separating controller/physics issues from later middleware effects.

ROS2 integration then introduced a deliberately more realistic runtime architecture. PID and LQR completed the full experiment with negligible controller computation cost. The model-selected MPC, despite its validated Part 1B numerical implementation and successful direct-Isaac transfer, became unstable as a runtime process: aggressive corrective effort, growing state error, constraint activity, solver cost, and missed updates interacted until the intended 500 Hz control sequence was no longer maintained.

A limited sensitivity study showed that increasing the MPC input penalty and reducing the control horizon could restore practical execution without shortening the 320 ms prediction horizon. The $\rho_u = 120$, $N_c = 20$ case completed the full 4 s run with all 2000 logged updates and a sub-2 ms 99th-percentile computation time in the retained data. This is reported as a practical runtime configuration, not a globally optimized MPC design. The incomplete MPC runs remain valuable as failure-mechanism evidence, but their tracking errors are not interpreted as valid controller-performance results.

The central outcome is methodological. Model validation, controller design, independent physics validation, and runtime validation answer different questions and should not be collapsed into one benchmark number. Likewise, the numerical ordering of PID, LQR, and practical-MPC RMSE in the ROS2 runs is not treated as a controller ranking because Part 2 did not perform a common task-level re-optimization. The next stage can now move from controller transfer to task-oriented robotics: planning and executing a manipulation task while treating tracking, timing, constraints, robustness, and task success as a coupled engineering problem.

A. Recorded ROS2 Runtime Summary

The table below reproduces the metrics generated directly from the CSV files included with this Overleaf bundle. It is intended to make the numerical basis of the report explicit and easy to regenerate. For incomplete MPC runs, RMSE is retained as a descriptive property of the recorded failure trajectory only; those rows are not used as controller-performance benchmarks.

Case	Samples	RMSE [deg]	Mean [ms]	P99 [ms]	Max [ms]
PID	2000	1.0063	0.00164	0.00416	0.0362
LQR	2000	2.3445	0.00728	0.01510	0.0386
MPC 10/20	186	14.1810	62.1549	160.0363	163.8499
MPC 10/40	159	10.6936	69.7548	166.8002	179.3156
MPC 120/20	2000	2.4865	0.75066	1.63587	2.90065
MPC 120/40	1730	3.3893	3.20570	36.9413	70.4231

B. Finalization Checklist

The technical narrative and recorded-data analysis are complete for the present scope. The following items remain optional finalization checks rather than silently invented details:

- Add the exact CPU model and OSQP version if a formal hardware/runtime reproducibility statement is desired.
- Decide whether to perform 2–3 repeated runs of the practical $\rho_u = 120$, $N_c = 20$ configuration for a quantitative repeatability table.
- If OSQP iteration history is required, re-run one representative failing and practical MPC case with iteration count logged into telemetry/CSV.
- Replace or supplement development screenshots with final publication-quality Isaac/ROS2 architecture screenshots if desired.
- Update repository paths if the public GitHub directory names change.